

PMap in Kotlin

or

How to make friends and influence people with functional programming and concurrency

A Good Day



Wesley Hall Tuesday at 10:33 AM

Page load that was 50+ seconds is now 9 second on our side.

David I owe you a drink, luckily James says there is a tab on the 25th so you can have my share 😊

Brilliant mate, thank you.



3



Johannes Merkle Yesterday at 10:00 AM



Dave! Amazing!



Julia Birkett 10:16 AM

this makes me wonder... parallelism was there in the document we built 4 years ago, and the effort was pretty low to do it.. why did we take so long to explore that?



David Wickes

Greetings! Good News! That degenerate case is now down to 6s

From a thread in # pegasus-snapp-collaboration | Sep 16th |

[View reply](#)

my bet is on lack of knowledge and people not being comfortable with parallelism and not seeing this as a quick exploration

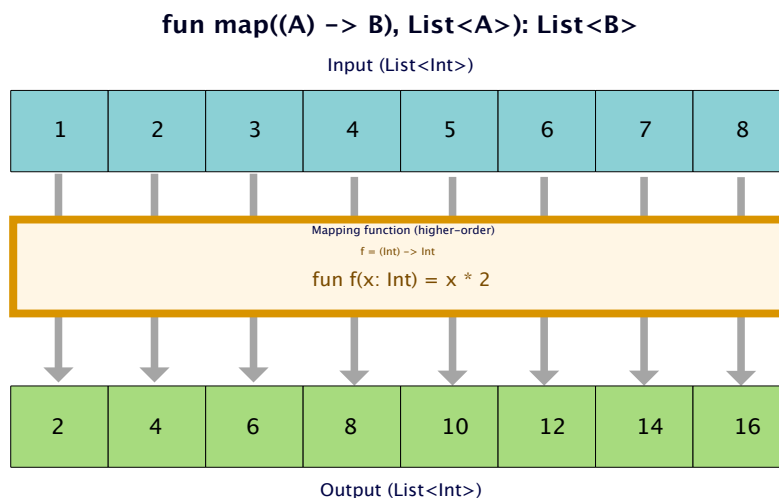
It was a good day on Monday. Not only did I get to indulge my favourite hobbies - functional programming and concurrency - but I also made my product owner happy, other engineers happy, and possibly even real users of Springer Nature happy.

One Weird Trick

"But how, Dave?" I hear you cry. Well, I'm going to tell you. But first, have you heard the good news about functional programming? It's getting to be old news these days, but I still get excited by it.

Specifically, today, `map`

`map`

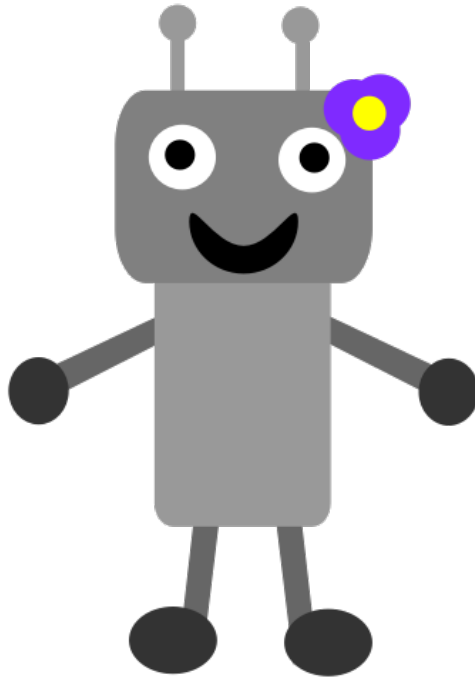


Map is definitely one of your entry level functional-ish programming constructs. Look - higher-order functions! This is an abstract picture of it with some beautiful type signatures so that we feel clever.

But, to help us understand what happens later on, let's break `map` right down.

This

is a robot. In your computer it's the current process, but don't think about that. Just focus on the pretty robot. Let's call her Rosie.



So with `map`, the program is like a tiny robot in a little factory, and it's doing a job. With `map`, that job is taking a big row of boxes, and turning it into a new row of boxes, while doing *something* to what's in the row.

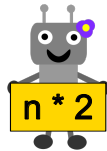
If we imagine that the job it to take some numbers and multiply them by two, it would look like this

So the little robot goes to the first box, gets the number out, multiplies by two, and then puts it in the first box of the output boxes.

(functional programming is really just working in a production line prove me wrong).

Input (List<Int>)

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---



?	?	?	?	?	?	?	?
---	---	---	---	---	---	---	---

Output (List<Int>)

does some work

Input (List<Int>)

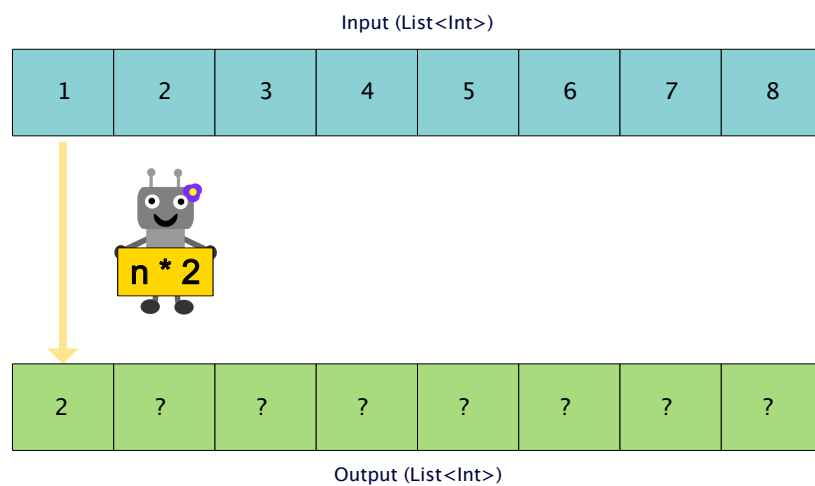
1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---



?	?	?	?	?	?	?	?
---	---	---	---	---	---	---	---

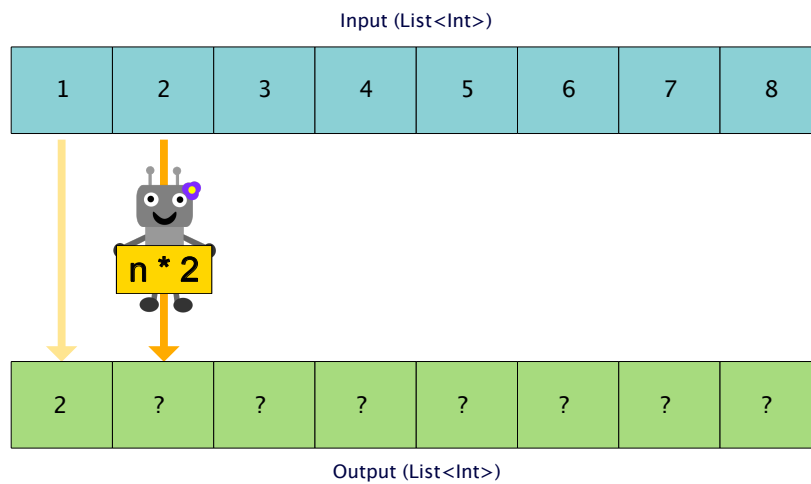
Output (List<Int>)

and then puts it in the right box on the output side

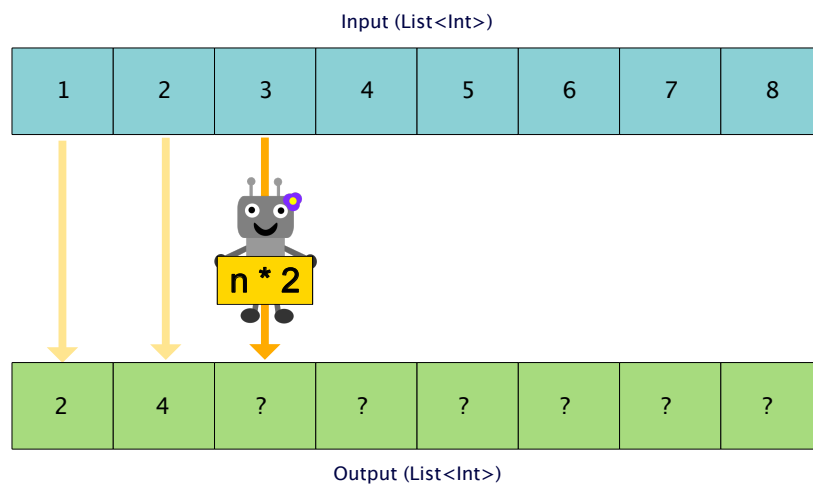


moves on to the next one

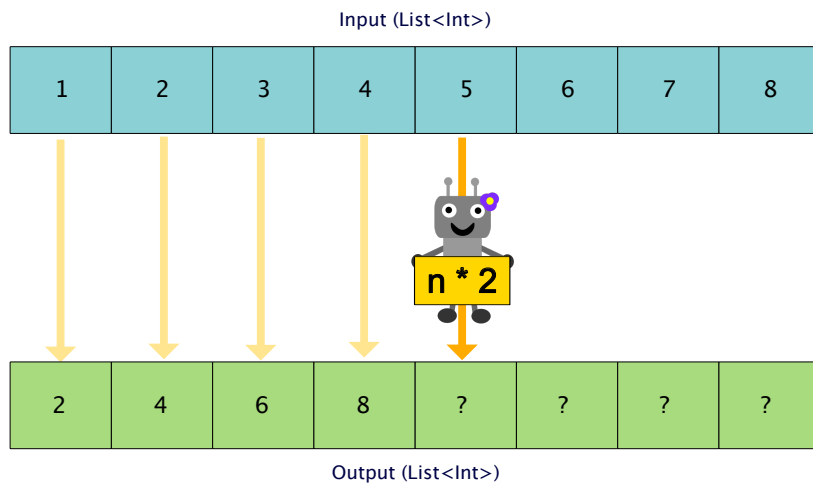
and does some work



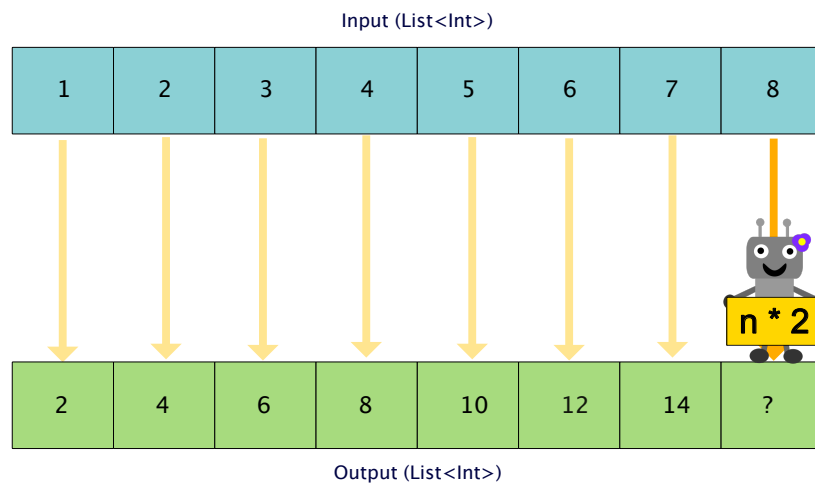
and so on



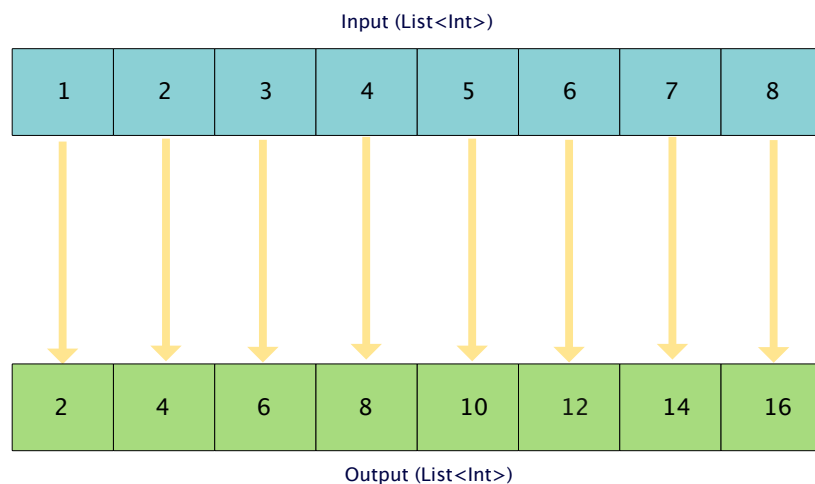
and so on



until it gets to the last number to process



and then it's done



Now before we start to dig into the fun stuff, I need to introduce or remind you of three rules for `map`

Rules For `map`

1. Size of output is same as size of input
2. We don't change the input
3. The function we map with should be "pure".

This last one is the important one.

3. The function we map with should be "pure".

The function doesn't care about the other items in the list. It's pure.¹

1. And if it's not pure, you're probably doing it wrong and should feel at least a little bit bad.

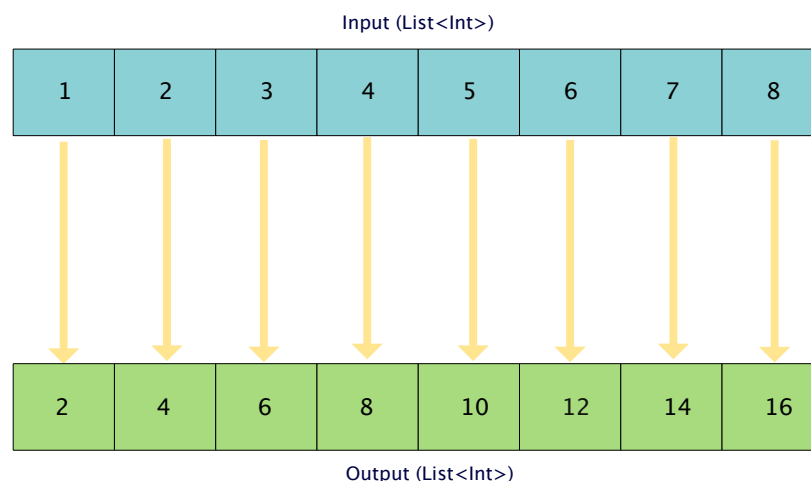
But what if it's... slow?

Standard `map` is great, but it has a one-track mind. It processes one item, then the next, then the next... sequentially.

If your transformation is quick, who cares? But what if each operation is a network call? Imagine you need to fetch details for 450 submissions. Each call is a snappy 20ms.

So look at the diagram, and imagine if each of those steps takes "some time".

But what if it's... slow?



If your transformation is quick, who cares? But what if each operation is a network call? Imagine you need to fetch details for 450 submissions. Each call is a snappy 20ms.

450 submissions × 20ms = 9000ms. That's a **9 second** wait. Not so snappy now, is it?

The total time is the **sum of all the individual operations**. That's our performance wall.

Now this is *kinda* what was happening with the Submission system when Researcher Home asked for a summary of a users submissions. We'd go to "a magic" ask it for a list of submission ids for that user. Then, with those ids, we'd `map` over them and ask for the details of the submission, *and* then we'd `map` again and ask for some more information about that submission from a few other boxes. But because those boxes were slow sometimes, and because the user somehow had 450 submissions, this could get *really slow*.

Hope through purity

But the good news here is that *we are using* `map`. Because, as I said `map` has some `_really` useful properties, and the key one for us is the purity property of the function - each individual

Well, that's Embarrassing...

Because each operation in a `map` is off in its own little world, we stumble upon a wonderful property: it's an **Embarrassingly Parallel** problem.

The name is perfect. It's so easy to break the work up into parallel tasks, it's almost embarrassing.¹ This is the whole reason `pmap` can even exist.

Embarrassingly Parallel (*adj.*): A problem so straightforward to solve with concurrency that explaining it to another developer feels awkward, as if you're stating the obvious.

1. And it saves us from writing all that horrible, error-prone boilerplate for managing threads ourselves. A win-win.

`pmap` : An Nice Abstraction

So `pmap` is a function that leverages the *purity* of the `map` function, and allows us to *magically* turn a sequential operation basically for free.

Prior Art: `pmap` Elsewhere

This isn't some new-fangled idea I just cooked up. We're treading a well-worn path here. Languages built for concurrency have had this for years.

Clojure

`pmap` is a core library function, often a drop-in replacement for `map`.

```
:: Sequential
(map inc [1 2 3 4])

;; Parallel
(pmap inc [1 2 3 4])
```

Scala

Scala achieves this by converting a collection to its parallel equivalent with `.par`.

```
// Sequential
List(1, 2, 3, 4).map(_ + 1)

// Parallel
List(1, 2, 3, 4).par.map(_ + 1)
```

Rust

Rust's ecosystem has the `rayon` crate, which provides a `.par_iter()` method to parallelize iterators.

```
// Sequential
vec![1, 2, 3, 4].iter().map(|n| n + 1).collect::<Vec<i32>>();

// Parallel (with the rayon crate)
use rayon::prelude::*;
vec![1, 2, 3, 4].par_iter().map(|n| n + 1).collect::<Vec<i32>>();
```

Python

Python's `multiprocessing` module can distribute work across a pool of processes.

```
# Sequential
def increment(n):
    return n + 1

list(map(increment, [1, 2, 3, 4]))

# Parallel
from multiprocessing import Pool

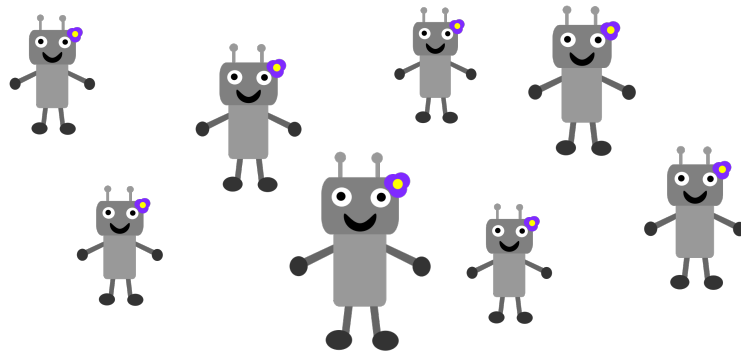
with Pool() as p:
    p.map(increment, [1, 2, 3, 4])
```

`pmap` is the magic word we'll use instead of `map` to make things faster. But to explain *how* we make things faster, let's go back to the robots.

Back to the robots

So if *one* robot was going to take a long time to do our job, how do we make it faster. Well, one option would be to build a faster robot. Which is actually a really good idea and is definitely something we could consider. Another would be to make the problem easier for our robot to solve. That's usually what we do. But if that's not going to happen then the only alternative is

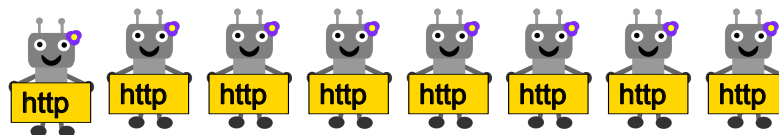
MOAR ROBOTS



so we throw eight robots at the problem

Input (List<Int>)

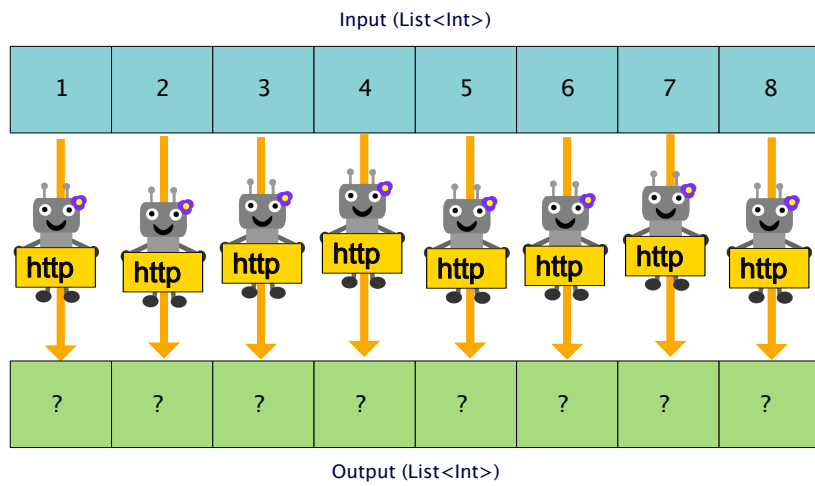
1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---



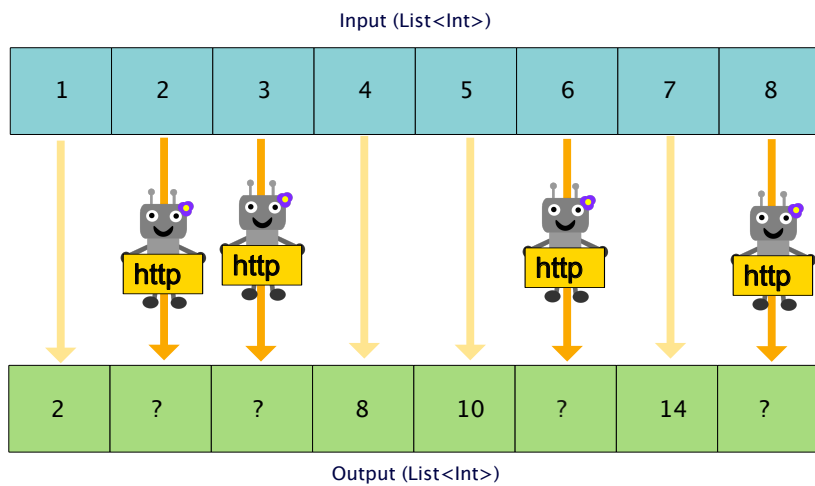
?	?	?	?	?	?	?	?
---	---	---	---	---	---	---	---

Output (List<Int>)

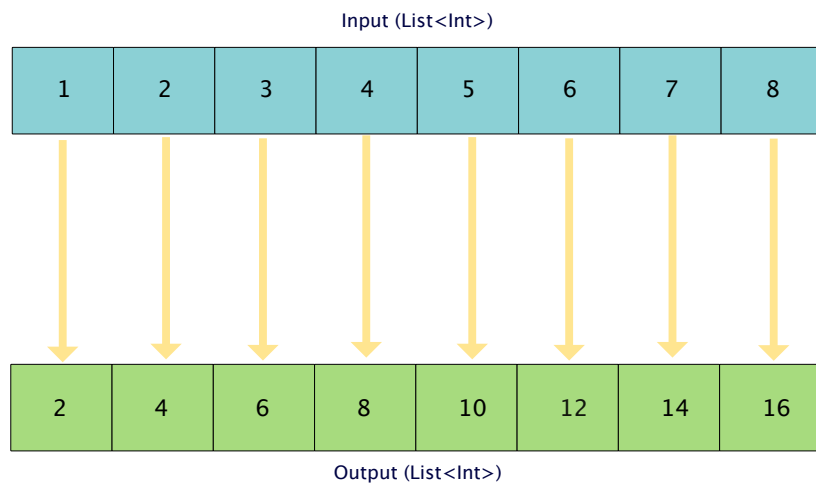
and hey look, those robots can now work on everything at the same time!



Some of them will finish before the others



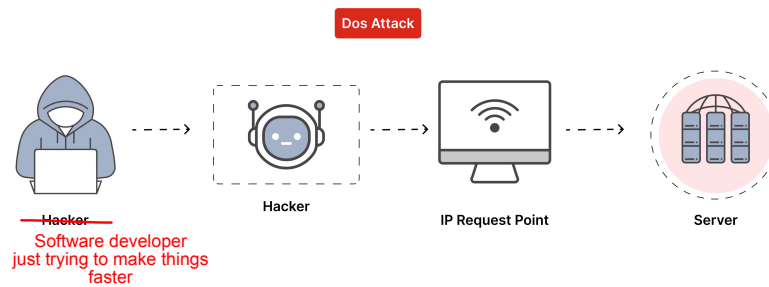
But the result is the same - every number has been mapped. And the whole operation will have taken only as long as the *slowest* operation.



All the robots, all the time?

But wait there maybe we should think about other people. For a change.

Don't DoS Yourself



In the above example I put one robot on every job that needed doing. If we kept doing this (which we can call *unbounded concurrency*, it would mean that we could have, say, 450 robots making http calls all at once to a web server. Now this *might* be OK - but more than likely if you're doing this you might just kill the server you're calling because you've just accidentally performed a denial of service attack on them, or run out of http connections on your side.

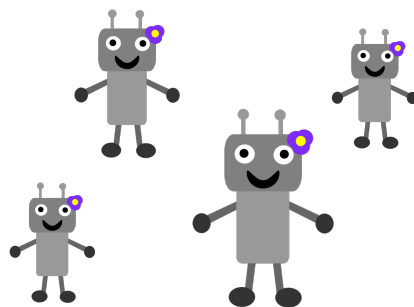
Pro tip: if you're do this accidentally, just tell everyone that you were performing a load test.

So what you need is a way of controlling how many jobs you're running at once. And *one* good way of doing this would be to *limit the number of robots*.

So let's have only four robots instead

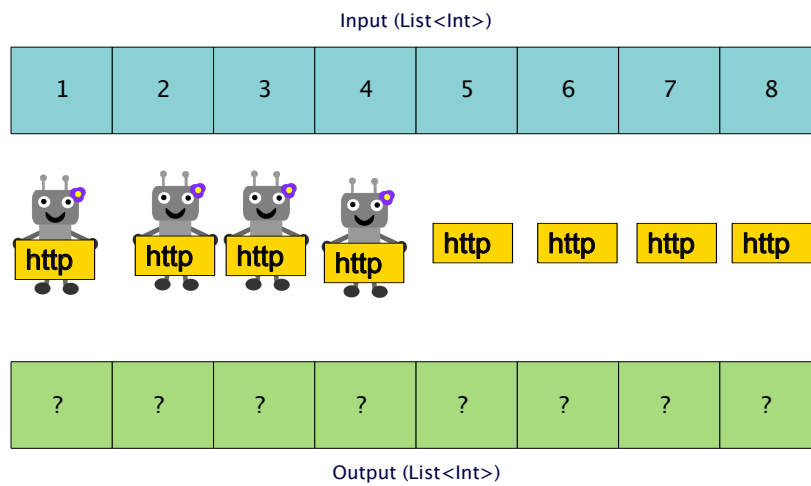


sorry, sorry

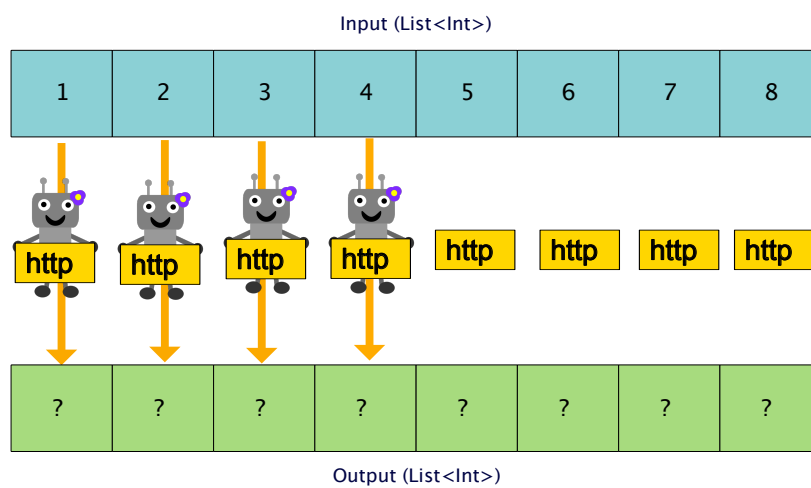


that's better

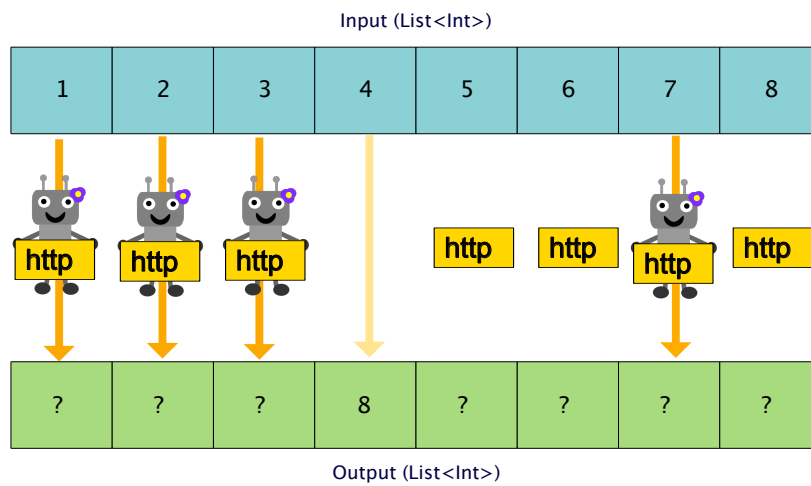
Now they can only 'pick up' the first four jobs that need doing



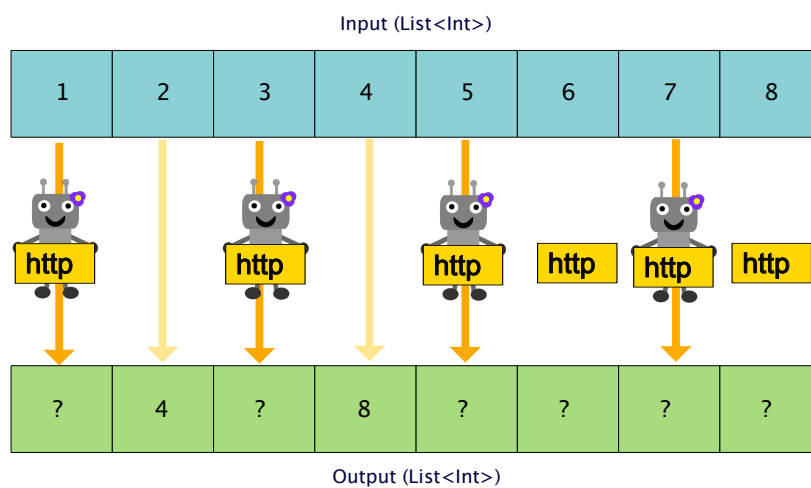
off they go. And when one of them finishes with their job...



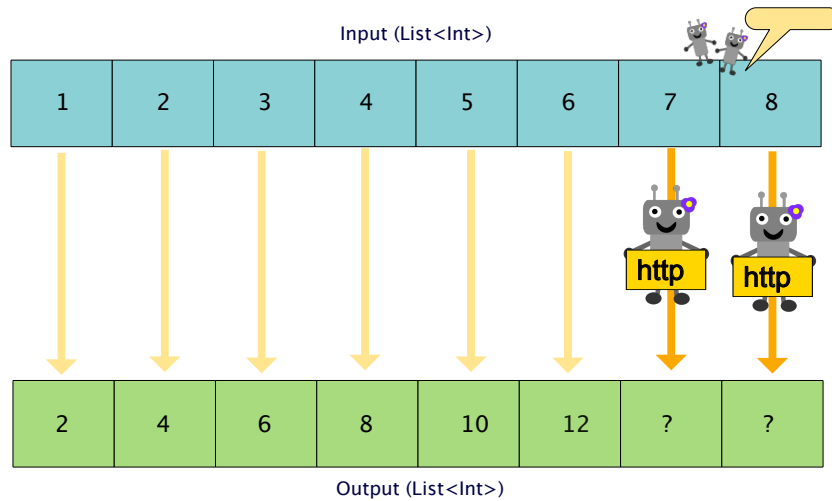
they just go to another of the available jobs, it doesn't matter which one.



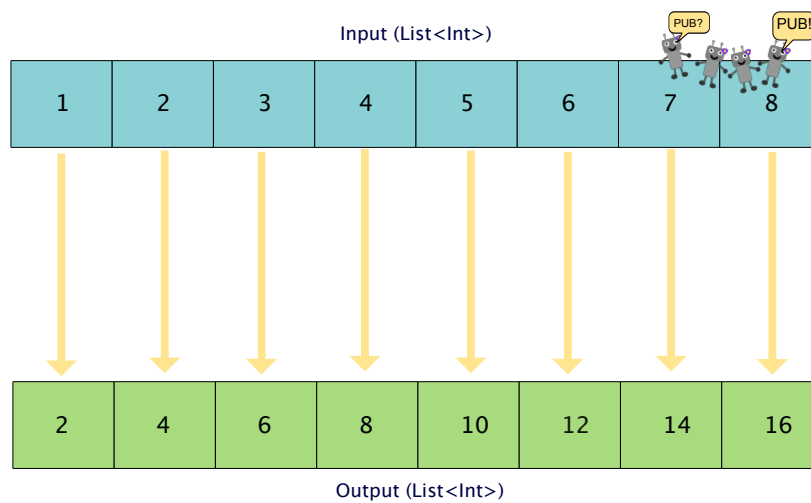
And the next one that finishes moves on to the next job



If there's nothing left to do they go and... relax?



Until finally it's all done and the robots... retire? Idk, the metaphor breaks down here.



But as you've seen, at no point did the number of jobs running *concurrently* exceed the number of robots. This is a great technique to limit the work in progress - in our example, the number of open http connections - at any one time.

So let's try and see what that looks like in our Kotlin code.

But what if... it goes wrong?

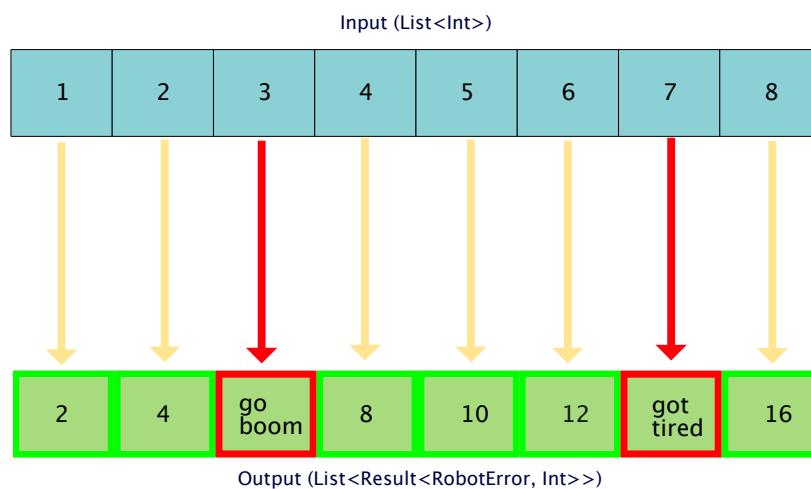
If any of those robots has a malfunction, what do we want to have happen.

Well, there are options

We could bring the whole `map` production line to a crashing halt. But that seems a bit extreme when there's only *one* robot gone wrong. What a waste of time and effort for the other robots.

We could ignore the failure, and just not put the number that went wrong in our list. But that's a bit smelly, because now we've got a `map` that doesn't return the same number of elements as went in.

So instead what we do is put our number that we're returning in a box which can have two different colours - red or green - to say whether something went wrong. And in the green boxes we've got our numbers, and in the red ones we've got a message about what went wrong with that number



So the first thing to mention is that we try to keep the signature as close to `map` as we can so that we get a drop-in replacement - it should be easy to use, so we use it.

Our Kotlin `pmap` : The Signature

```
private fun <T, R> Iterable<T>.pmap(  
    maxConcurrency: Int,  
    transform: (T) -> R,  
) : List<Result<PMapError, R>>
```

It's an extension function, it takes a `maxConcurrency` limit (which is the number of robots available in the above example), and it returns a list of `Result`s, because we do still need a way to communicate that things have gone wrong.

So now on to the implementation:

Core Concept #1: Robots Virtual Threads

The engine of our `pmap` is `Executors.newVirtualThreadPerTaskExecutor()`.

This is the magic from Project Loom. Virtual threads are incredibly cheap, so we can spin one up for every single item without breaking a sweat. It's perfect for tasks that spend most of their time waiting for a network call to return. These are our robots who are going to do the work for us.

```
private fun <T, R> Iterable<T>.pmap(...) {  
    val semaphore = Semaphore(maxConcurrency + 1)  
    + val executor = Executors.newVirtualThreadPerTaskExecutor()  
  
    try {  
        val futures: List<Future<Result<PMapError, R>>> = map { item ->  
    +     executor.submit<Result<PMapError, R>> {  
            // ...  
        }  
    }  
    // ...  
    } finally {  
    +     executor.shutdown()  
    }  
}
```

The scariest bit here is probably the signature `List<Future<Result<PMapError, R>>>` - the key bit of this is the `Future` - which is just an object that represents something that may or may not have happened yet. What we're doing here is just creating a list of things that haven't happened yet. More on that soon.

Core Concept #2: ~~Number of Robots~~ Semaphore

So why not run a million things at once? Because that's how you get rate-limited, crash your database, or generally make a nuisance of yourself. And so we stop this by limiting the number of robots who are allowed to play at any one time.

A Semaphore is our friendly bouncer, making sure only `maxConcurrency` tasks get into the club at any one time.

```
private fun <T, R> Iterable<T>.pmap(...) {  
+   val semaphore = Semaphore(maxConcurrency + 1)  
   val executor = Executors.newVirtualThreadPerTaskExecutor()  
  
   try {  
       val futures: List<Future<Result<PMapError, R>>> = map { item ->  
           executor.submit<Result<PMapError, R>> {  
+               semaphore.acquire()  
               try {  
                   // ...  
               } finally {  
+               semaphore.release()  
               }  
           }  
       }  
       // ...  
   } finally {  
       executor.shutdown()  
   }  
}
```

Core Concept #3: ~~Red and Green Boxes~~ The Result Type

And instead of our boxes, we use the `Result` type. All the results of our `transform` will, if they throw, turn into a `Result`. And we have a special error type to say that the problem came up in the `map`.

```
private fun <T, R> Iterable<T>.pmap(  
    maxConcurrency: Int,  
    transform: (T) -> R,  
+): List<Result<PMapError, R>> {  
    // ...  
    try {  
        val futures: List<Future<Result<PMapError, R>>> = map { item ->  
            executor.submit<Result<PMapError, R>> {  
                // ...  
                try {  
+                Success(transform(item))  
+                } catch (e: Throwable) {  
+                Failure(PMapError("Exception during pmap: $e", e))  
                } finally {  
                    // ...  
                }  
            }  
        }  
+    return futures.map { it.get() }  
    } finally {  
        // ...  
    }  
}
```

The Full Implementation

[on GitHub](#)

So here it is, all the pieces together in one place. No magic, just a few good ideas working in concert.

```
private fun <T, R> Iterable<T>.pmap(
    maxConcurrency: Int,
    transform: (T) -> R,
): List<Result<PMapError, R>> {
    val semaphore = Semaphore(maxConcurrency + 1)
    val executor = Executors.newVirtualThreadPerTaskExecutor()

    try {
        val futures: List<Future<Result<PMapError, R>>> = map { item ->
            executor.submit<Result<PMapError, R>> {
                semaphore.acquire()
                try {
                    Success(transform(item))
                } catch (e: Throwable) {
                    Failure(PMapMError("Exception during pmap: $e", e))
                } finally {
                    semaphore.release()
                }
            }
        }
        return futures.map { it.get() }
    } finally {
        executor.shutdown()
    }
}
```

Demo

[on GitHub](#)

Enough talk. Let's see this thing in action.

We'll simulate a slow network call and pit `map` against `pmap` in a fight to the death. Or, you know, just time them.

Gotchas & Considerations

Before you go and `pmap` all the things, a few words of warning...

1. It's not always faster

For tiny lists or trivial functions, the overhead of starting that virtual thread executor pool might actually make `pmap` slower than `map`. Don't use a sledgehammer to crack a nut.

2. Keep it pure. NO REALLY.

If your transform function messes with shared state, you're entering a world of pain. Race conditions are not your friend. If you need to do this, don't use `pmap`. And good luck, you're on your own.

3. Don't DDOS yourself

That `maxConcurrency` value is important. Be a good citizen and respect your downstream services.

4. Only Works For I/O Bound Operations

I've not talked about this at all, but bear in mind that this `pmap` will *not* improve the speed of an `map` transformation time is mostly taken up with your CPU doing work. That's because of the way that the virtual threads work.

But the good news is that almost everything we do that's slow is slow because of I/O - input/output. Http calls, your database, writing files, etc. So this `pmap` is a good fit.

If you *do* need to optimise a CPU-bound operation, you'll need your own `pmap` that uses real threads.

Go now and read some blog posts about the difference between concurrency and parallelism, and how threads work.

Conclusion

So there you have it. `pmap` is a powerful, ergonomic, and surprisingly simple tool to have in your belt. It's a great example of how a good abstraction can make a hard problem... well, not *easy*, but at least manageable. And a lot more fun.

Questions?
